

Financial Control Tower

An evidence-backed, policy-gated decision layer
for merchant payments

Razorpay Buildathon 2026 — Open Track

Technical report

5th Sep 2026

Abstract

Merchant payment tooling is overwhelmingly *detect-only*: it reports what happened and leaves the merchant to work out what to do. Financial Control Tower closes that gap with a ten-stage loop — observe, remember, detect, investigate, simulate, recommend, approve, act, verify, learn — implemented as service modules inside a single Next.js application. Against a deterministic synthetic batch of 10,000 payment attempts, the system detects 37 anomalies, opens investigations on 29, and surfaces ₹8.42L of revenue at risk of which ₹5.17L is modelled recoverable and ₹3.84L has been recovered. The flagship case — a UPI degradation concentrated on one issuer — carries ₹4.82L of exposure across 1,284 payments, with success rate falling from 97.8% to 81.4% and the issuer's share of failures rising from 21% to 87%, a 4.1× concentration. Before recommending anything, the system prices three mutually exclusive responses, including doing nothing, on expected recovery, expected cost and net expected benefit. The chosen action is bounded by a deterministic policy engine (₹5,000 per customer, two attempts, 500 customers, human approval above low risk), executed only by a server-side executor that re-derives its own cohort, verified against the ledger, and written to an audit trail. We deliberately demonstrate a failure: the retry path is halted by the gateway and then by the retry ceiling, and the agent declines to retry further. That path, not the success path, is the strongest evidence the system is safe to give agency to.

1 Problem statement

The Buildathon's Open Track asks for a product built on Razorpay's platform that solves a real merchant problem. The problem we chose is not the absence of data. Merchants already have dashboards, settlement reports, refund logs and payment records. The problem is that all of it is *detect-only*, and detection is the cheap half of the work.

A detect-only tool tells a merchant that the payment success rate fell to 81%. It does not tell them:

- **Where the loss is concentrated.** An 81% success rate merchant-wide could be a broad platform issue or a single issuer failing on a single rail. The response differs completely.
- **What it is worth.** 1,284 failed payments is a count. ₹4.82L of exposure, of which ₹3.14L is modelled recoverable, is a decision.
- **What the alternatives cost.** A retry is cheap and converts poorly on a degraded rail. Routing customers to another

rail converts better and costs an outbound contact. Doing nothing is free and loses ₹2.10L. Ranking these requires all three to be priced on the same terms.

- **Whether it is safe to act.** The moment software can move money or contact customers, the interesting question stops being “is the model right?” and becomes “what is the worst thing this can do if it is wrong?”

The gap between the fourth bullet and the first three is why most “AI for payments” demos stop at generating text. Generating a recommendation is easy. Making a recommendation that is bounded, reversible, auditable and refusable is the actual engineering problem, and it is what this report is about.

A second, quieter failure of detect-only tooling is that thresholds are global while merchants are not. A 3.7% refund rate is healthy for one catalogue and alarming for another; Sunday revenue at a third of a weekday is a crisis for one merchant and structural for another. Every detection in this system is a comparison against a baseline learned from *that merchant's* own history (Section 10), which is what keeps the alert list to three items instead of thirty.

2 System architecture

The product is a single Next.js 16 application using the App Router. There is no separate backend, no microservice boundary, no message broker and no worker process. Server Components read the domain modules directly; Route Handlers expose the same modules over HTTP for the interactive paths. This is a deliberate constraint: a judge should be able to run `npm install && npm run dev` and have a working product, and the same repository should deploy to Vercel unchanged.

The ten stages of the loop are ordinary TypeScript modules under `lib/`. Figure 1 shows the loop with each stage mapped to its module.

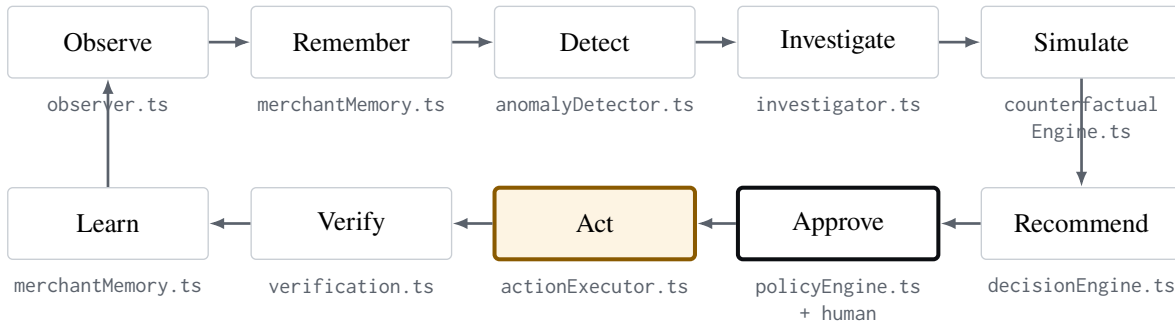
2.1 Stage responsibilities

OBSERVE — `observer.ts`

Reduces the ledger to ten scalar signals (failure rate, success rate since onset, issuer share of failures, method mix, per-SKU refund rate, checkout conversion, settlement variance, settlement delay, overdue receivables, chargeback exposure), each with its window and sample size. It measures; it decides nothing.

REMEMBER — `merchantMemory.ts`

Loads seven learned baselines from 42 days of history, each



Act is the only stage permitted to reach a payment gateway.

It is unreachable without a passing policy decision and, above low risk, a named human.

Figure 1: The ten-stage loop. Each stage is a module under `lib/`, not a service. `Act` is the only stage with gateway access, and it is unreachable without a passing policy decision and, above low risk, a named human approval.

with a confidence, an observation count, a source and a plain-language statement of why the system believes it.

DETECT — `anomalyDetector.ts`

Emits an anomaly where an observation has left its learned band *by enough to matter financially*. Every anomaly carries an impact in rupees, a modelled recoverable amount, a confidence and an affected count.

INVESTIGATE — `investigator.ts`

Assembles evidence, segments the cohort, states a root cause with a mechanism, and — importantly — records the competing hypotheses it tested and why each was rejected.

SIMULATE — `counterfactualEngine.ts`

Scores every candidate response, including inaction, on the same three quantities (Section 5).

RECOMMEND — `decisionEngine.ts`

Converts the winning scenario into a *bounded action plan*: an explicit cohort, a per-customer ceiling, an attempt ceiling, a risk rating and a named policy.

APPROVE — `policyEngine.ts` + **UI**

Deterministic rule evaluation, then a human where the policy demands one.

ACT — `actionExecutor.ts`

The only module permitted to call a gateway. Runs in five observable stages.

VERIFY — `verification.ts`

Re-reads the outcome against the ledger and against the guardrails the action was approved under. A partial verdict is a legitimate result.

LEARN — `merchantMemory.ts`

Baselines are recalibrated on verified outcomes; the recalibration itself is an audit event.

2.2 State and determinism

The read path is a pure function of a single seed (`DEMO_SEED = 20260218`) and a fixed clock (`2026-02-18T16:45:00+05:30`). `Math.random()` does not appear anywhere in the codebase; all randomness comes from a seeded `mulberry32` generator. The dataset is built once per process and cached on `globalThis`, so it survives module reloads and is identical across processes, machines and deployments.

Write state — which actions the operator approved, executed or rejected, and the audit events those produced — is held client-side behind a `useSyncExternalStore`. This is not a short-

Entity	Rows	Role
Merchant	1	Tenant root
Customer	3,200	Repeat-purchase signal
Product	8	Per-SKU refund analysis
Transaction	10,000	The ledger
PaymentEvent	derived	Lifecycle per transaction
Refund	333	Refund-spike detection
Chargeback	9	Dispute exposure
Settlement	14	Reconciliation
CheckoutSession	742	Funnel drop-off
Subscription	240	Recurring failure risk
Invoice	118	Receivables ageing
Anomaly	7	Detection output
Investigation	4	Evidence-backed case
Evidence	10–12	Traceable support
TimelineEntry	4–6	What ran, when
Scenario	3/case	Priced alternatives
Recommendation	1/case	Selected scenario
Action	9	Bounded plan
ActionResult	1/exec	Outcome + verification
AuditEvent	31+	The chain of record
MerchantMemory	7	Learned baselines
Policy	1	RECOVERY_V2 · 2.3

Table 1: Entities and their cardinality in the seeded batch. Row counts are the values the running application reports at `/api/diagnostics`.

cut: it keeps the server read path referentially transparent, it survives a serverless cold start on Vercel where a module-level mutation would not, and it makes *Reset demo* exact rather than approximate.

3 Data model

Twenty-two entities span a ledger layer and an intelligence layer. Figure 2 shows the relationships; Table 1 lists the entities with their role and cardinality in the seeded batch.

The schema is expressed canonically in `prisma/schema.prisma`, and `lib/types.ts` mirrors it field for field. The demo materialises it in memory rather than in Postgres — a deliberate choice, since requiring a database

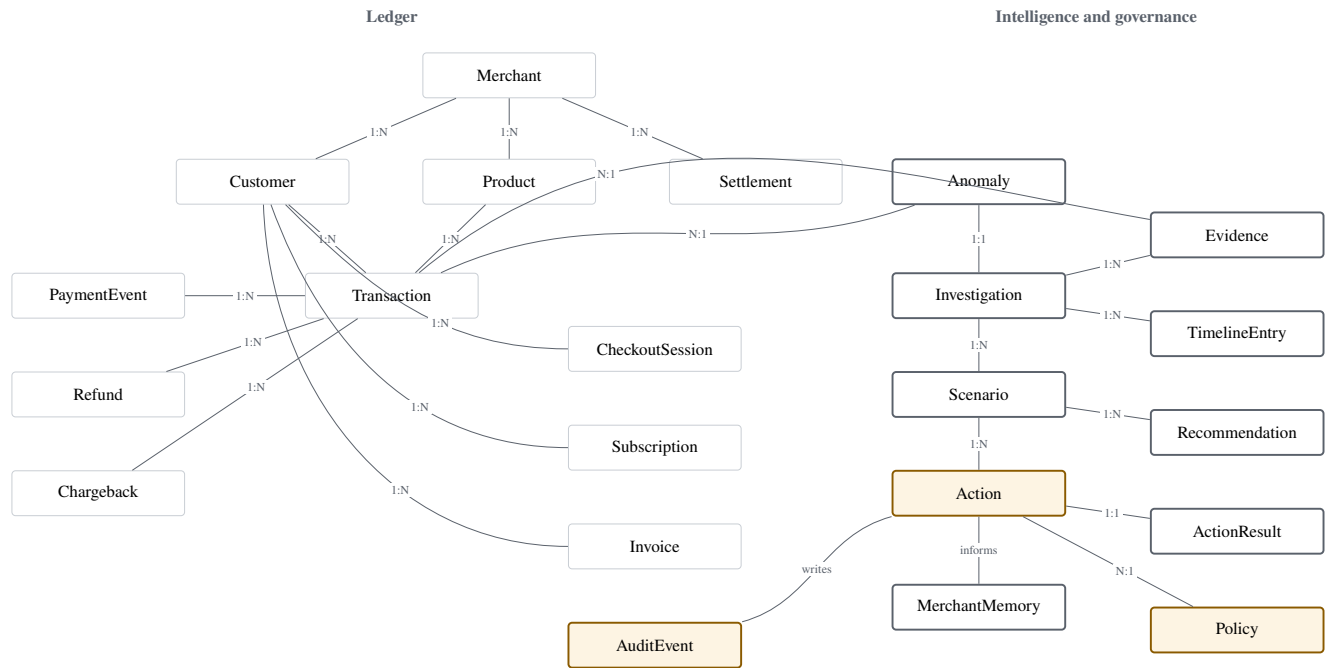


Figure 2: Entity relationships. The ledger layer (left) is what a payment platform already has. The intelligence and governance layer (right) is what turns it into decisions, and every entity in it is traceable back to transactions.

to see the product would defeat the point — but the two are the same shape, so moving to a persistent deployment is a repository-layer change and not a UI change.

Two fields on *Transaction* deserve mention because the rest of the system leans on them. *anomalyId* attributes a transaction to a detected anomaly, which is what makes “1,284 affected payments” a query rather than a claim. *recoveryProbability* is a per-transaction model output, present only on failures, and is the basis of every “recoverable” figure in the product.

4 Detection and investigation

4.1 The synthetic batch

The dataset covers 24 hours ending at the fixed clock and is *constructed* to its targets rather than sampled and hoped over. Amounts are scaled so that cohort totals are exact (*scaleToTotal*, and *scaleToTotalCapped* where a policy ceiling must hold for every member by construction). Segment shares are struck by even stride rather than Bernoulli sampling, because Bernoulli sampling leaves an 87% target sitting at 84% and a report should not have to round.

The batch is shaped in three windows:

Window	Attempts	Failures	Success
00:00–09:12 (pre)	2,382	52	97.8%
09:12–16:45 (event)	7,618	1,415	81.4%
of which 14:30–16:10	1,099	494	55.1%
Day total	10,000	1,467	85.3%

Captured value is ₹18,42,000 across 8,533 payments, an average order value of ₹216. The affected cohort averages ₹375 — a 1.7× skew that the investigation surfaces as evidence in its own right, since higher-value collect requests failing first

is consistent with issuer-side risk checks queueing behind a saturated service.

4.2 The five required anomaly classes

1. **UPI payment degradation** (*inv_1042*, critical) — ₹4.82L across 1,284 payments, 91% confidence.
2. **Refund spike on one SKU** (*inv_1043*, watch) — Aurora Buds Pro, 3.7% → 8.9%, ₹1.10L of excess refund value over nine days, 88% confidence.
3. **Checkout conversion drop** (*inv_1044*, watch) — 93.8% → 91.8%, 61 abandoned sessions, ₹72K, 76% confidence.
4. **Settlement discrepancy** (*inv_1045*, watch) — ₹38K unrec-onciled on 1 of 14 cycles, 84% confidence.
5. **Recoverable failed payments** (opportunity) — 184 high-intent customers, ₹1.92L exposure, ₹1.26L modelled recoverable.

Two further findings — receivables ageing (₹82K across 27 invoices) and chargeback exposure (₹58K across 9 disputes) — complete the ₹8.42L risk rollout. The rollout is a sum over the anomaly ledger, and the *recoverable failed payments* entry is explicitly excluded from it because it is a view onto the UPI cohort rather than a separate exposure. Double-counting exposure is the easiest way for a product like this to lie, so the exclusion is enforced in code (*DERIVED_ANOMALIES*) rather than by convention.

4.3 From evidence to root cause

The investigation workspace is deliberately not a chat interface. It assembles ten evidence items for the UPI case, each weighted:

- six *transaction* items — the highest-value failures, each opening a drawer with the full payment-event trail, customer history and error code;
- three *aggregate* and *timeseries* items — issuer concentration, method/device cut, and the 20-minute success-rate series;

- one *comparison* item — the order-value skew.

The root cause is stated with a mechanism, not just a label: collect requests to the issuer are timing out before customer approval, which the gateway reports as a timeout rather than a decline. It is supported by a two-bar comparison of the issuer’s baseline failure share (21%) against its incident share (87%).

What makes this an investigation rather than an assertion is the record of *rejected* hypotheses:

Hypothesis	Rejected because
Merchant-side check-out regression	Card success is 97.6% and netbanking 97.1% today; a checkout regression would not spare two rails
Customer balance or auth failures	86% of affected error codes are timeouts or gateway errors; declines are flat
Traffic shift toward UPI	UPI’s share of attempts is 61% against a 62% baseline
Android release regression	Android carries 88% of failures but 58% of all traffic; iOS payments on the same issuer fail at the same elevated rate

Table 2: Alternative hypotheses tested and dismissed for `inv_1042`. A confidence score without this table is a number without a method.

Confidence is reported at two levels and they are not the same number: 91% on the root cause, 82% on the recovery estimate. Conflating them would overstate how much is known about the money.

5 The counterfactual engine

This is the component that separates a decision system from an alerting system, and it is a safety property before it is a feature.

Every candidate response, *including doing nothing*, is scored on the same three quantities:

$$\begin{aligned}
 R &= V_{\text{risk}} \times r_{\text{conv}} && \text{expected recovery} \\
 C &= c_{\text{unit}} \times n_{\text{attempts}} && \text{expected cost} \\
 B &= R - C && \text{net expected benefit}
 \end{aligned}$$

with inaction scored as a loss, $L = V_{\text{risk}} \times r_{\text{churn}}$, so that $B_{\text{do nothing}} = -L$. Table 3 gives the scored alternatives for the UPI case, where V_{risk} is ₹4,82,000.

Scenario	Recovery	Cost	Net
A. Do nothing	—	—	−₹2.10L
B. Retry failed payments	₹1.70L	₹12K	₹1.58L
C. Alternate rail offer	₹2.40L	₹28K	₹2.12L

Table 3: Counterfactuals for `inv_1042`. Conversion is modelled at 35.3% for a retry, because it lands on the same degraded rail, and 49.8% for the alternate rail. Retry cost is ₹4.67 per attempt over two attempts across 1,284 payments; outreach cost is ₹21.80 per contacted customer. Scenario C wins on net expected benefit and is recommended.

Three properties matter more than the arithmetic.

Inaction is priced. A merchant who declines every recommendation should still be told what that decision costs. Presenting a loss of ₹2.10L next to a gain of ₹2.12L turns “do nothing” from a default into a choice.

The comparison is made before the recommendation, not after. The decision engine does not generate a plan and then justify it; it takes the argmax of B over the scored set. The recommendation text is derived from that ranking, which is why it can cite the runner-up (“₹2.12L net against ₹1.58L for a retry on the same degraded rail”).

Each scenario states its assumptions. Conversion rates, unit costs and the policy ceilings that bound the scenario are shown on the card. A merchant who disagrees with the 49.8% conversion assumption can see it and say so, which is not possible when the model emits only a conclusion.

The recommended scenario is then *narrowed* into an action. Scenario C covers all 1,284 payments; the action targets only the 184 high-intent customers — one failed attempt each, all priced under the policy ceiling — for ₹1.42L of expected recovery. A first tranche that a human can inspect is worth more than a complete plan that a human cannot.

6 Decision safety architecture

The single most important design decision in this product is stated negatively: **the language model cannot execute a financial action**. Not “is instructed not to” — cannot. The boundary is structural, and it is enforced in four places.

1. The model’s output surface is a closed set. The only structured output a model may produce is a `scenarioKey` drawn from the three scenarios the counterfactual engine already scored, plus prose. The Zod schema (`11mRecommendationSchema`) admits `do_nothing` | `retry` | `alternate_method`, a headline, a reason and a confidence. There is no field for an endpoint, an amount, a cohort, a customer list or a policy name. A model that emits one is rejected at parse time and the deterministic engine’s choice stands.

2. The executor takes an identifier, not an instruction. `runStage(actionId, stage, constraints)` looks the plan up server-side and re-derives the cohort from the ledger. The HTTP body carries a stage name and, optionally, constraints. Nothing else crosses the boundary.

3. Constraints can only narrow. An operator may reduce a cohort or lower a ceiling from the UI. The executor applies `Math.min(plan, request)` on both, so a client that requests a larger cohort or a higher ceiling *tightens* the action rather than widening it. Attempting to widen is not an error condition to be handled; it is structurally impossible to express.

4. Credentials refuse live mode. `assertTestMode` rejects any key that does not begin with `rzp_test_`, and it is called on the authorisation header path, not only at configuration time. There is no code path in the repository that can authenticate against live mode.

To this we add a fifth property that is not about the model at all: the product never claims an external call that did not happen. Without credentials, gateway responses are labelled `MOCK_*` and their messages say, in the response body that the UI renders, that no request was sent to Razorpay. Actions that need no gateway — a refund hold, a checkout reordering, a

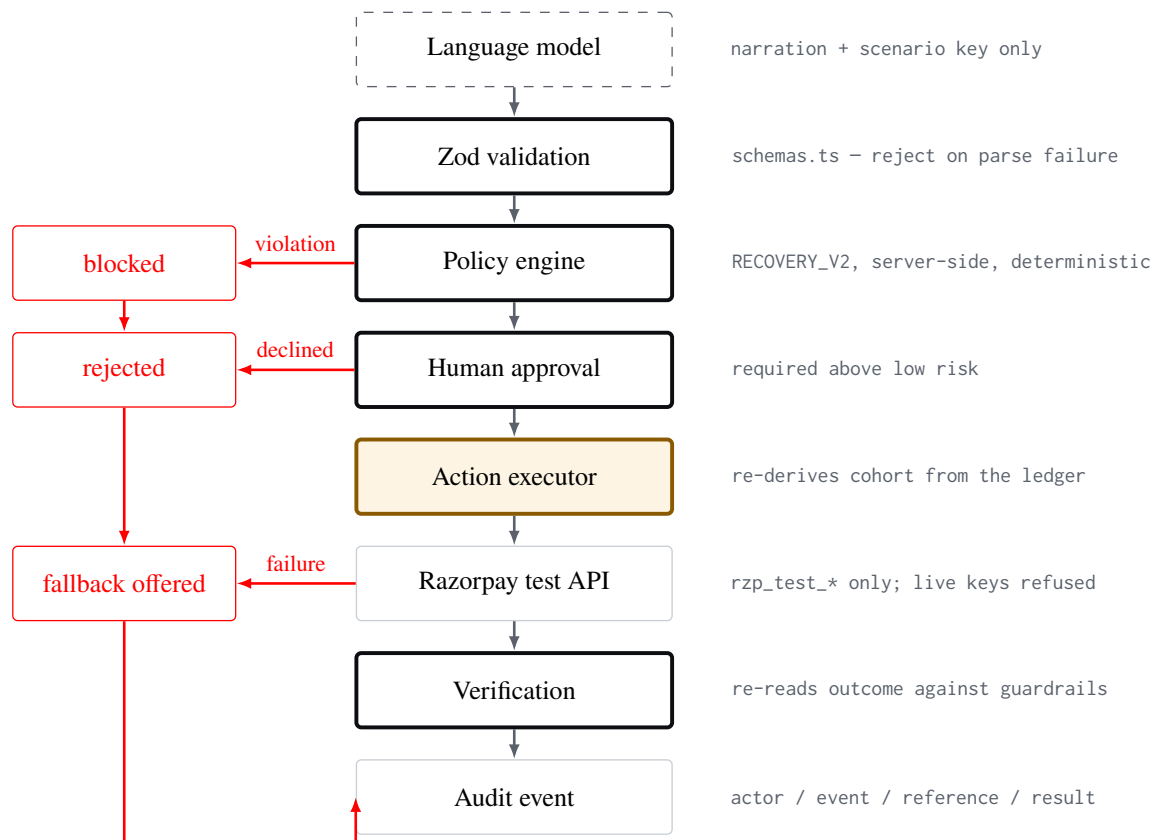


Figure 3: The mandatory pipeline. Red branches are terminal outcomes that are themselves audited: a policy block, a human rejection and a gateway failure all end in an audit event rather than in silence.

reconciliation filing — report “Gateway: not required for this action class” rather than manufacturing a call.

7 Policy engine

The policy engine is a pure function of an action plan and a policy. It contains no model output, no probability and no I/O. It runs server-side on *every* stage of an execution, not once at approval, so an action cannot become non-compliant part-way through a run.

The ceilings are visibly load-bearing rather than decorative. The six largest failures in the incident total ₹59,880 and every one of them exceeds ₹5,000, so no automated action may touch them; they are shown on the action page under *Held for manual approval* with their transaction identifiers. The historical action ledger contains `act_2026`, a 640-customer retry that was blocked before approval by `MAX_CUSTOMERS_PER_ACTION` and re-scoped to 500, and `act_2028`, a 15% blanket discount rejected by the merchant. Of 143 actions executed across the batch, 12 were blocked by policy and 37 required a human approval.

8 Graceful failure handling

An always-succeeds demo proves nothing about production-readiness. The interesting question about an agent is not what it does when the world cooperates. The system therefore ships a deterministic failure and treats it as a first-class path.

`act_2040`, “Retry failed UPI payments”, targets the same 184-customer cohort as the recommended action. It has already

spent one of its two automatic attempts during the incident — which is visible in the payment-event trail of every affected transaction — so it is *allowed* to run, and it does.

1. **PREPARE.** Cohort re-derived from the ledger: 184 targets, ₹1,92,000 exposure.
2. **POLICY.** Five of five rules pass. `MAX_RETRY_ATTEMPTS` reports “1 of 2 attempts used” — the action is compliant.
3. **ELIGIBILITY.** 184 of 184 eligible; 6 larger failures worth ₹59,880 remain outside automation.
4. **GATEWAY.** A recovery order is created, then queried for captured payments. None is found inside the attempt window. This is an *observed negative result*, not a fabricated one: in Live Test Mode it is a real `GET /v1/orders/:id/payments` returning an empty items array.
5. **Policy re-evaluation.** The executor re-runs the policy with `attemptsUsed + 1`. `MAX_RETRY_ATTEMPTS` now fails. Further retries are halted.
6. **VERIFY** is never reached. The run is terminal.

The UI does not crash, does not spin and does not show a stack trace. It shows:

Action could not be completed

Reason. Temporary payment failure.

Policy. Retry ceiling reached — 2 of 2 attempts already spent on this cohort.

AI response. “I will not repeatedly retry this payment. The configured policy limit has been reached, and a further attempt against the same degraded rail would add cost without adding expected recovery. An alternate recovery workflow is available.”

and a single button, *Review alternative*, which routes to

Rule	Value	Rationale
MAX_AUTO_ACTION_AMOUNT	₹5,000	Bounds the worst case of a single mis-targeted action to a sum the merchant can absorb and reverse
MAX_RETRY_ATTEMPTS	2	Repeated retries on a degraded rail add cost, irritate customers and can trip issuer risk rules; two attempts capture nearly all recoverable value
MAX_DISCOUNT_PERCENT	10%	Beyond this, recovery costs more than the order earns
HIGH_RISK_REQUIRES_APPROVAL	true	Customer-contacting and money-moving actions carry reputational consequences the model cannot price
MAX_CUSTOMERS_PER_ACTION	500	Caps how many customers one mistake reaches before a human sees a result
TEST_MODE_ONLY	true	No live-money credential is loadable here

Table 4: RECOVERY_V2 · 2.3. Each threshold answers “what is the worst outcome if this action is wrong?”, not “what is the best outcome if it is right?”

act_2041. Three audit events are written: the gateway call, the TEMPORARY_PAYMENT_FAILURE, and the policy activation of the fallback. The failure is not a dead end and it is not hidden; it is a transition with a record.

The fallback then completes. Against a modelled ₹1,42,080, the executed alternate-rail action recovers ₹1,41,175 across 130 of 184 customers, and verification passes all five checks. Reporting a realised figure that differs from the model, rather than echoing the estimate back, is the point: a system that always “recovers exactly what it predicted” is not measuring anything.

9 Audit trail design

Every event carries the same tuple:

(at, actor, event, refType, refId, result, detail)

with actor $\in \{\text{ai, merchant, api, policy, system}\}$ and result $\in \{\text{ok, failed, blocked, info}\}$. The stream is filterable by actor and by failure, and events generated during the current session are visually distinguished from the seeded history.

The design goal is reconstruction without context. An auditor reading only the audit trail can answer: what was detected, and on what evidence; who or what opened the investigation; what root cause was concluded; what was recommended and why it was held for approval; who approved it; which policy passed and when; which endpoint was called and with what response; what failed; which rule halted it; what was offered instead; what was recovered; and whether the result was verified. Table 5 is that chain for the demo run.

Seeded history contributes a second complete chain from 12 February — including its own partial failure, fallback and post-

recovery memory update — so the audit view is informative before a presenter touches anything.

10 Merchant memory

Seven baselines are learned from 42 days of history. Each carries a value, a confidence, an observation count, a source, and a written explanation of why the system believes it (Table 6).

Memory does three jobs. It **contextualises**: 8.9% is meaningless in isolation and unambiguous against a 3.7% baseline held for 33 consecutive days. It **suppresses**: Sunday revenue at a third of a weekday is structural for this merchant, and encoding that prevents a weekly false positive. And it **discriminates**: because UPI’s share of *attempts* is unchanged at 61% while UPI *fulfilment* has collapsed, the investigation can rule out a customer-behaviour explanation and say so in the summary.

Baselines are recalibrated on verified outcomes. The 12 February recovery closed with “Recovery model recalibrated on 96 new observations”, written to the audit trail — learning is an auditable event, not an invisible drift.

11 Results

Table 7 reports the seeded batch. These are the figures the running application computes; GET /api/diagnostics returns them live.

The batch is a synthetic test environment and the report labels it as such everywhere it appears. What the numbers demonstrate is internal consistency: the ₹8.42L rollup is the sum of the anomaly ledger; the ₹4.82L is the sum of 1,284 transaction amounts; the ₹3.14L is the amount-weighted expectation $\sum_i a_i p_i$ over those transactions; and the ₹1,41,175 realised figure is the sum of the amounts of the 130 cohort members whose deterministic draw fell below their uplifted recovery probability. No displayed figure is a constant typed into a component.

12 UI and UX design rationale

The visual direction — restrained neutral surfaces, hairline borders, a single blue accent used sparingly, tabular figures throughout, monospace reserved for identifiers and timestamps, high information density — is not a style preference. It follows from two requirements.

Trustworthiness. This product asks a merchant to approve an action that contacts 184 of their customers. Interfaces that signal excitement — gradients, glow, oversized numbers, animated emphasis — are precisely wrong for that request. The design borrows the register of infrastructure documentation and trading terminals because those are the interfaces people trust with consequential decisions.

Legibility of comparison. Almost every screen is a comparison: observed against baseline, scenario against scenario, realised against modelled. Density serves that directly — the counterfactual cards are readable side by side without scrolling, the memory grid shows seven baselines at once, and the audit table puts actor, event, reference and result on one line. Whitespace is spent on separating comparisons, not on decorating them.

Time (IST)	Actor	Event	Reference	Result
14:32:11	ai	AI detected payment anomaly	anm_upi_degradation	info
14:33:04	ai	Investigation created	inv_1042	ok
14:34:12	ai	Payment methods segmented	inv_1042	info
14:35:01	ai	Root cause identified	inv_1042	ok
14:36:20	ai	Revenue impact calculated	inv_1042	info
14:36:48	ai	Counterfactuals scored	inv_1042	info
14:37:02	ai	Recovery recommendation generated	act_2041	ok
14:37:03	policy	Approval requirement applied	RECOVERY_V2	blocked
16:45:30	merchant	Action approved	act_2040	ok
16:45:33	policy	Policy validation passed	RECOVERY_V2	ok
16:45:36	system	Transaction eligibility checked	act_2040	ok
16:45:41	api	Gateway called	act_2040	ok
16:45:41	api	Payment retry failed	act_2040	failed
16:45:42	policy	Fallback policy activated	RECOVERY_V2	blocked
16:47:00	merchant	Action approved	act_2041	ok
16:47:11	api	Recovery offers dispatched	act_2041	ok
16:47:27	ai	Result verified	act_2041	ok

Table 5: The demo chain end to end, as rendered at /audit. Rows above 16:45 are produced by the detection pipeline before any operator input; rows from 16:45 are produced by the executor during the session. The two bold rows are the failure and the policy response to it.

Baseline	Normal	Current	Status
Payment failure rate	2.1%	14.7%	Unusual
Issuer concentration	21%	87%	Unusual
UPI mix	62%	61%	Normal
Refund rate (SKU-2481)	3.7%	8.9%	Unusual
Sunday revenue	₹7-9L	₹8.2L	Normal
Settlement timing	1.3 d	1.4 d	Normal
Checkout conversion	93.8%	91.8%	Drifting

Table 6: Learned baselines. The *Normal* rows are as important as the unusual ones: they are what stops the detector raising an anomaly every Sunday morning.

Concrete consequences: monospace for every identifier (TXN_82931, act_2041, UTR6704666335) so they are recognisable as machine values; tabular figures so columns of money align; a two-pixel status rule at the top of severity cards instead of a coloured background; animation confined to execution progression and status transitions, where it carries meaning. Loading states are skeletons of the eventual layout; error states name what could not be read (“Unable to retrieve this transaction”) and offer a retry; no error surface renders a stack trace.

13 Tech stack and deployment

Next.js 16 (App Router), React 19, TypeScript in strict mode, Tailwind CSS, Recharts for the time-series and volume charts, hand-drawn SVG for sparklines, Lucide for iconography, Zod for validation. Server Components read the domain modules directly; Route Handlers cover the interactive paths; server-only guards the gateway client and the executor at the module level so a client import is a build error rather than a security incident.

The whole thing deploys with vercel or a GitHub import. There is no Python, no FastAPI, no Docker, no Redis, no

worker and no second deployment target. Every route handler is a serverless function and every page is server-rendered on demand.

Every environment variable is optional. With an empty environment the application runs in full Demo Mode: deterministic data, deterministic detection and scoring, a labelled local gateway adapter. Supplying a Razorpay test key pair switches the badge to *Live Test Mode* and makes the executor issue real calls against the test environment. Supplying a model key switches the investigation summary from the built-in deterministic text to model-written prose; nothing else changes, because nothing else is allowed to depend on the model.

That property is a judgeability argument as much as an engineering one. A judge with no keys, no database and a slow network still sees the complete product, and can tell at a glance which mode it is in, because the badge says so on every screen.

14 Limitations and future work

We are explicit about the boundary between what is built and what a live-money deployment would require.

Synthetic data. Every figure describes a generated ledger. No real merchant or customer data is present. A production deployment would read from the merchant’s own transaction and settlement records through the platform APIs.

Recovery is modelled, not collected. In Demo Mode “recovered” is a deterministic outcome over the seeded cohort. In Live Test Mode the API calls are real but create test-mode order objects; the recovery value is still modelled, and the result message says so. Actually moving money would require payment links, customer consent flows and reconciliation against real captures.

The recovery model is calibrated, not trained. Per-transaction probabilities are drawn from a bounded distribution and normalised so the amount-weighted mean matches a target. A production version would fit on the merchant’s own historical

Measure	Value
Transactions analysed	10,000
captured / failed	8,533 / 1,467
Captured value	₹18,42,000
Average order value	₹216
Anomalies detected	37
Investigations run	29
Revenue at risk	₹8.42L
Potential recovery	₹5.17L
Actual recovered	₹3.84L
Actions executed	143
Actions blocked by policy	12
Human approvals	37
<i>Flagship case inv_1042</i>	
Affected payments	1,284
Revenue at risk	₹4,82,000
Modelled recoverable	₹3,14,000 (65.2%)
Success rate, pre / event	97.8% / 81.4%
Acute-window trough	55.1%
Issuer failure share	21% → 87% (4.1×)
High-intent cohort	184 / ₹1,92,000
Held above policy ceiling	6 / ₹59,880
Executed recovery	₹1,41,175 (130/184)
against modelled	₹1,42,080

Table 7: The seeded batch. All values are computed from the generated ledger, not asserted; the generator is constructed to hit its targets exactly so that a presenter sees the same numbers on every run.

recovery outcomes, report a prediction interval rather than a point estimate, and be re-fit on every verified action.

Detection is baseline comparison. Thresholds against learned bands, not statistical process control or changepoint detection. This is deliberate for a demo where determinism outranks sophistication, but a production detector would want CUSUM or Bayesian changepoint methods with explicit false-positive budgets.

Persistence and multi-tenancy. Session state is per-browser and the ledger is per-process. Production requires the Prisma schema against Postgres, tenant isolation, and an append-only audit store with retention guarantees — an audit trail that can be edited is not an audit trail.

Approval identity. The demo records “merchant” as the approving actor. A real deployment needs authenticated users, role-based authority limits (who may approve above which value), and non-repudiation on the approval record.

Adversarial robustness. We constrain what the model can express, but we have not adversarially tested prompt injection through merchant-controlled data such as product names or refund reasons. The structural constraint — the model cannot name an endpoint or an amount — limits the blast radius by construction, which is the right defence, but it should still be tested.

15 Conclusion

Financial Control Tower is an argument that the hard part of applying AI to payments is not detection or explanation but *bounded agency*. The system detects anomalies against a merchant’s own learned baselines, investigates them into evidence-backed root causes with rejected alternatives on the record, prices every response including inaction on the same terms, and then subjects the winner to a deterministic policy engine, a human approval and a server-side executor that re-derives its own scope. It verifies what it did against the ledger and writes the whole chain — including its failures — to an audit trail.

The demonstration we are most confident in is the one that fails. A retry is approved, executes, is refused by the gateway, is then halted by its own retry ceiling, and the agent declines to try again and offers a different route instead. Every step is recorded. That behaviour, and not the ₹1.41L it recovers afterwards, is what would make a system like this deployable on a payments platform.

Most financial software tells merchants what happened. Financial Control Tower helps decide what should happen next — safely.

A Appendix A: audit log excerpt

Verbatim from /audit, showing the 12 February reference recovery. It contains its own partial failure and fallback, which is why it is retained as the calibration case for the recovery model.

```
11:14:22 ai AI detected payment anomaly anm_20260212 info
Netbanking degradation, 96 affected payments
11:20:03 ai Recovery recommendation generated act_2031 ok
Alternate payment offer for 96 customers
11:26:44 merchant Merchant approved action act_2031 ok
act_2031 approved - alternate payment offer, 96 customers
11:26:51 policy Policy validation passed RECOVERY_V2 ok
RECOVERY_V2 - 5 of 5 rules satisfied
11:27:09 api Razorpay test API called act_2031 ok
POST /v1/orders -> 96 recovery orders
11:27:40 api Payment retry failed act_2031 failed
11 of 96 returned TEMPORARY_PAYMENT_FAILURE
11:27:41 policy Fallback policy activated RECOVERY_V2 blocked
Retry ceiling reached for 11 payments; routed to the
alternate-rail workflow
11:31:09 api Payment recovered act_2031 ok
64 of 96 converted - INR 84,200 recovered
11:41:09 ai Result verified act_2031 ok
INR 84,200 recovered against INR 79,100 modelled - verified
11:41:30 ai Memory updated mem_failure_rate ok
Recovery model recalibrated on 96 new observations
```

B Appendix B: policy rule table

Rule	Value	Enforced in	Rationale
MAX_AUTO_ACTION_AMOUNT	₹5,000 / customer	evaluate → maxAmountPerCustomer	Bounds the worst case of a mis-targeted action to a reversible sum
MAX_RETRY_ATTEMPTS	2 / transaction	evaluate → attemptsUsed	Repeated retries on a degraded rail add cost and can trip issuer risk rules
MAX_DISCOUNT_PERCENT	10%	evaluate → discountPercent	Keeps recovery margin-positive
HIGH_RISK_REQUIRES_APPROVAL	true	evaluate → risk	Customer-contacting actions carry consequences the model cannot price
MAX_CUSTOMERS_PER_ACTION	500	evaluate → targetCustomers	Blast-radius ceiling before a human sees a result
TEST_MODE_ONLY	true	client.ts → assertTestMode	No live-money credential is loadable

C Appendix C: entity relationship diagram

The full-width version of Figure 2 appears in Section 3. The canonical definition, including every field, index and enum, is `prisma/schema.prisma` in the repository. The in-memory materialisation in `lib/demo/dataset.ts` produces exactly these shapes, and `lib/types.ts` mirrors them field for field, which is what makes a move to Postgres a repository-layer change rather than a rewrite.

D Appendix D: reproducing the figures

Quantity	Source
Batch shape, rates, rollups	GET /api/diagnostics
Incident statistics	lib/analytics/metrics.ts getIncidentStats()
Counterfactual scores	lib/ai/counterfactualEngine.ts upiScenarios()
Policy decisions	lib/policies/policyEngine.ts evaluate()
Execution outcome	POST /api/actions/act_2041/execute
Audit chain	GET /api/audit + session events

Reproduction. `npm install` && `npm run dev`, then open `http://localhost:3000`. No keys, no database, no containers. Every figure in this report is recomputed on each request from seed 20260218.